Name: Krish Singh
Roll: AID25072
B.Tech Artificial Intelligence & Data Science - 2nd Semester
Object Oriented Programming – Lab Assignment 10b

You should Observe:

1. Threads execute independently
2. Exception in one thread does not stop others
3. Exception handling must be done within thread
4. throws passes responsibility within thread context

Viva Questions:

1. Why do exceptions not propagate to the main thread?
Ans: Exceptions do not propagate to the main thread because each thread executes independently with its own call stack, so an exception occurring in one thread only affects that specific thread and does not impact or interrupt the execution of the main thread or other threads.

2. Difference between throw and throws in threading?
Ans: The throw keyword is used to explicitly generate an exception within a method or thread, whereas throws is used in a method declaration to indicate that the method may pass an exception to its caller; however, in multi-threading, even if throws is used, exceptions must still be handled within the run() method since there is no higher-level caller to catch them.

3. What happens if try-catch is removed from run()?
Ans: If the try-catch block is removed from the run() method, any unhandled exception will cause that particular thread to terminate abruptly, while other threads and the main program will continue executing normally, typically with the JVM printing a stack trace for the failed thread.

4. Why is start() preferred over run()?
Ans: The start() method is preferred over directly calling run() because start() creates a new thread and invokes the run() method internally, enabling concurrent execution, whereas calling run() directly executes it as a normal method in the current thread without creating any new thread.

5. Can multiple threads share the same exception handling logic?
Ans: Yes, multiple threads can share the same exception handling logic by using common Runnable classes, shared methods, or similar try-catch implementations, but even in such cases, the exception handling is performed independently within each thread’s execution context.

PART A: Exception Handling Fundamentals

Experiment A1: Basic Exception (try-catch)

Code:

```
class BasicException{
   public static void main(String[] args){
      int a=10, b=0;
      try{
         int result = a/b;
         System.out.println("result");
      }
      catch (ArithmeticException e){
         System.out.println("Error: Division by 0");
      }
   }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/
Error: Division by 0

Process finished with exit code 0
```

Task

•Change values of b

•Observe when exception occurs

•Identify type of exception

Experiment A2: Using throw (Manual Exception)

Code:

```
class UsingThrow{
   static void checkAge(int age){
      if(age<18){
         throw new ArithmeticException("Not Eligible");
      }
      System.out.println("Eligible");
   }
   public static void main(String[] args){
      checkAge(16);
   }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/JetBrains/Toolb
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : Not Eligible
    at UsingThrow.checkAge(UsingThrow.java:4)
    at UsingThrow.main(UsingThrow.java:9)

Process finished with exit code 1
```

Task
•Modify condition
•Create your own validation rule

Experiment A3: Using throws (Propagation)
Code:

```
class UsingThrowsPropagation{
   static int divide(int a, int b) throws ArithmeticException{
      return a/b;
   }
   public static void main(String[] args){
      try{
         int result = divide(10,0);
         System.out.println(result);
      }
      catch (ArithmeticException e){
         System.out.println(("Handled in main"));
      }
   }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/kr
Handled in main

Process finished with exit code 0
```

Task
•Remove try-catch and observe behavior
•Add multiple methods and propagate exception

Output after Task:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/JetBrains/Too
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
    at UsingThrowsPropagation.divide(UsingThrowsPropagation.java:3)
    at UsingThrowsPropagation.main(UsingThrowsPropagation.java:7)

Process finished with exit code 1
```

PART B: Threading Fundamentals

Experiment B1: Basic Thread using Runnable

Code:

```
class MyRunnable implements Runnable{
   public void run(){
      for(int i=1;i<=5;i++){
         System.out.println("Thread"+i);
      }
   }
}
public class BasicThreadUsingRunnable{
   public static void main(String[] args){
      Thread t = new Thread(new MyRunnable());
      t.start();
      t.run();
   }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/JetBrai
Thread1
Thread1
Thread2
Thread2
Thread3
Thread3
Thread4
Thread5
Thread4
Thread5

Process finished with exit code 0
```

Task
•Replace start() with run() and observe difference
•Add print statements in main
Output after Task:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/JetBrai
Thread1
Thread2
Thread3
Thread4
Thread5
Thread1
Thread2
Thread3
Thread4
Thread5

Process finished with exit code 0
```

Experiment B2: Multiple Threads

Code:

```
class MyRunnable2 implements Runnable{
    String name;

    MyRunnable2(String n){
        name =n;
    }
    public void run(){
        for(int i=0; i<5;i++){
            System.out.println(name+": "+ i);
        }
    }
}
public class MultipleThreads{
    public static void main(String[] args){
        Thread t1 = new Thread(new MyRunnable2("T1"));
        Thread t2 = new Thread(new MyRunnable2("T2"));
        Thread t3 = new Thread(new MyRunnable2("T3"));
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/JetBrains/Toolbox/apps/i
T1: 0
T1: 1
T1: 2
T1: 3
T1: 4
T3: 0
T2: 0
T3: 1
T2: 1
T3: 2
T2: 2
T3: 3
T2: 3
T3: 4
T2: 4

Process finished with exit code 0
```

Task

•Observe output order

•Explain why output is not sequential – because they run concurrently not sequentially